Deferred Rendering 을 이용한 렌더링 최적화

부경대학교 컴퓨터공학과 4 학년

안지우

Git: https://github.com/powercrabman

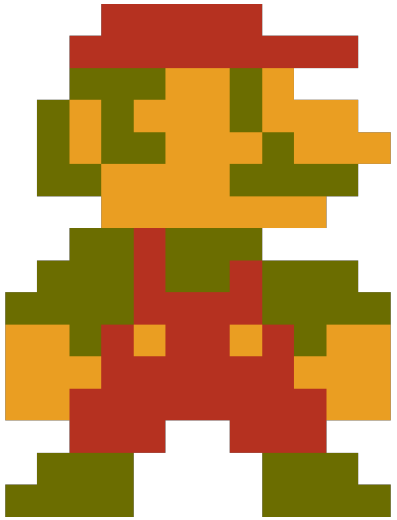
Blog: https://powerclabman.tistory.com/

배경지식 – 렌더링 기초

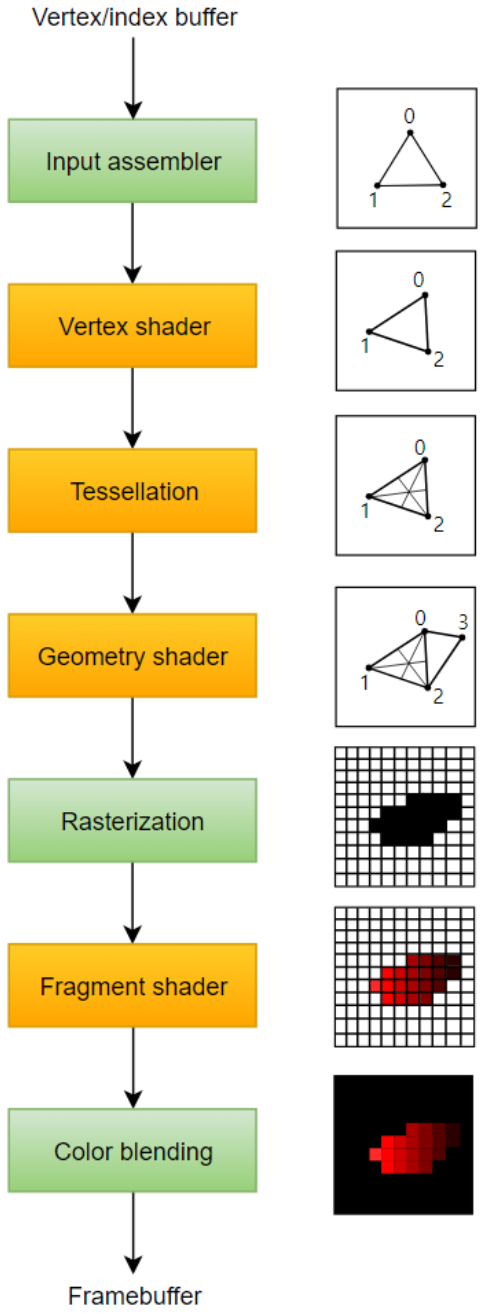
- 개념 : 스크린에 그림을 그리는 모든 행위
- 일반적으로 GPU 명령으로 수행된다 .



3D 렌더링

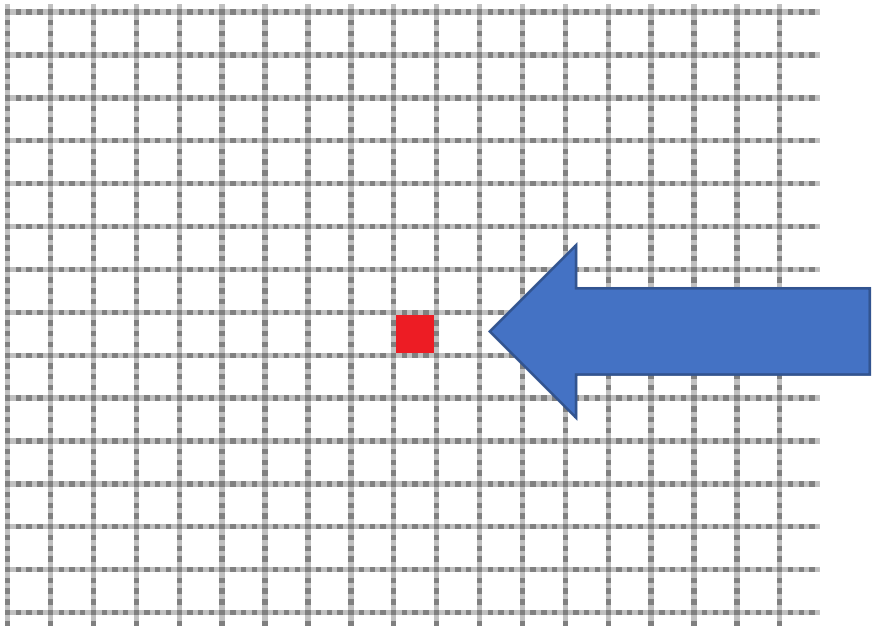


2D 렌더링



배경지식 – 렌더링 기초

- 2D 든 3D 는 결국 렌더링은 **픽셀**을 (엄청 많이) 찍는 행위
- 픽셀 하나를 찍을 수 있으면 그 어떤 그림도 렌더링할 수 있다 .

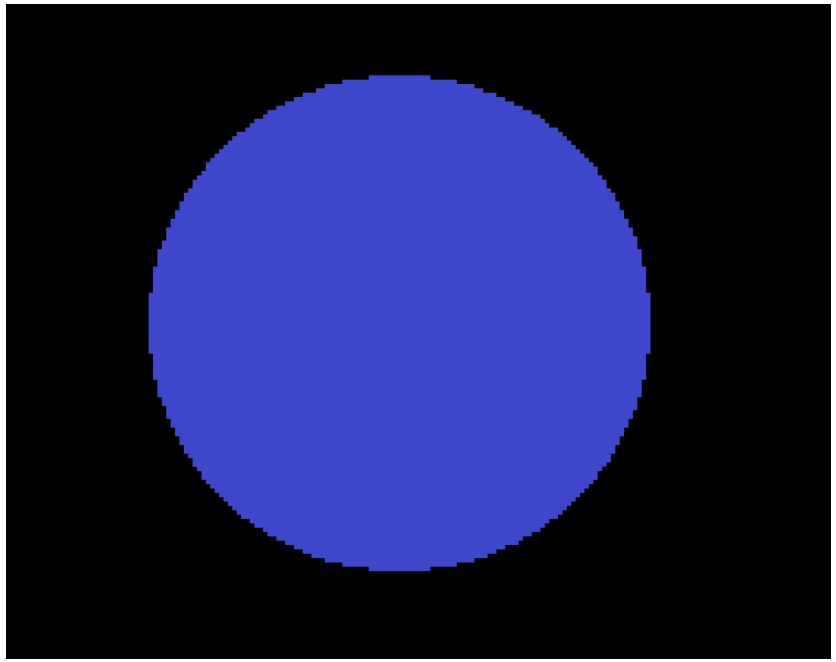


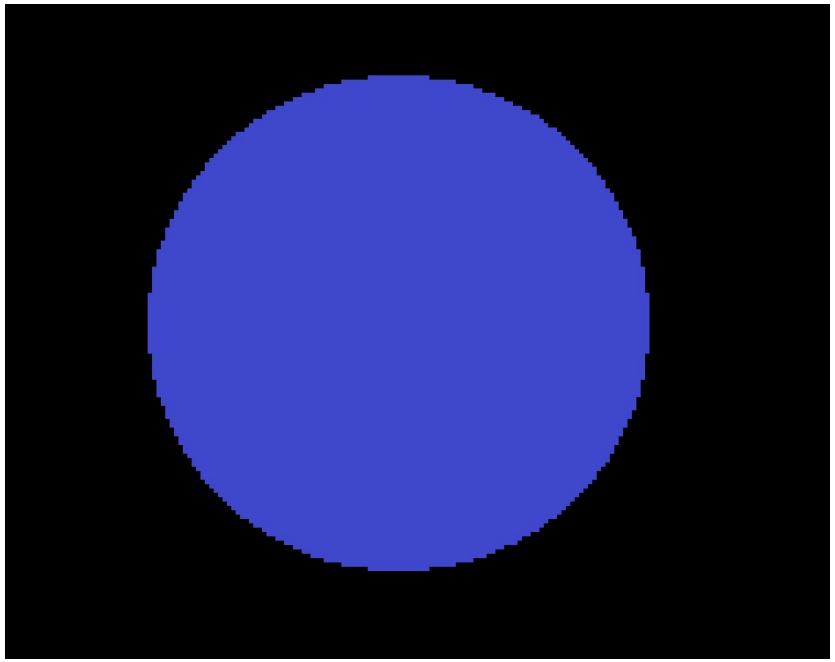
픽셀을 찍을 **위치 , 색** 을 결정할 수 있으면 픽셀을 찍을 수 있다 .

위치 (x, y): (10, 8)
색상 (r, g, b): (255, 0, 0)

배경지식 – 3D 렌더링 기초

- 3D 그림을 그리기 위해서는 몇 가지 정보가 더 필요한데…

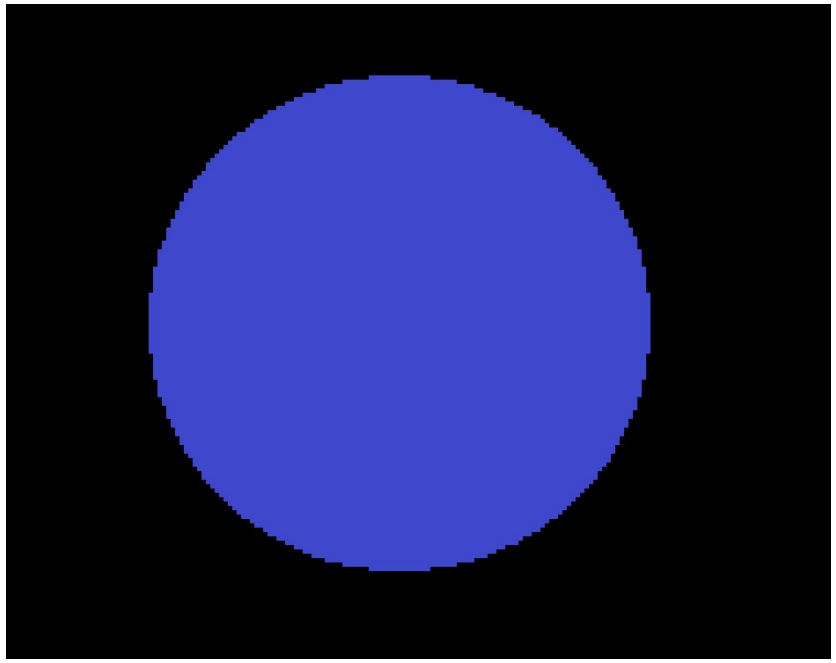


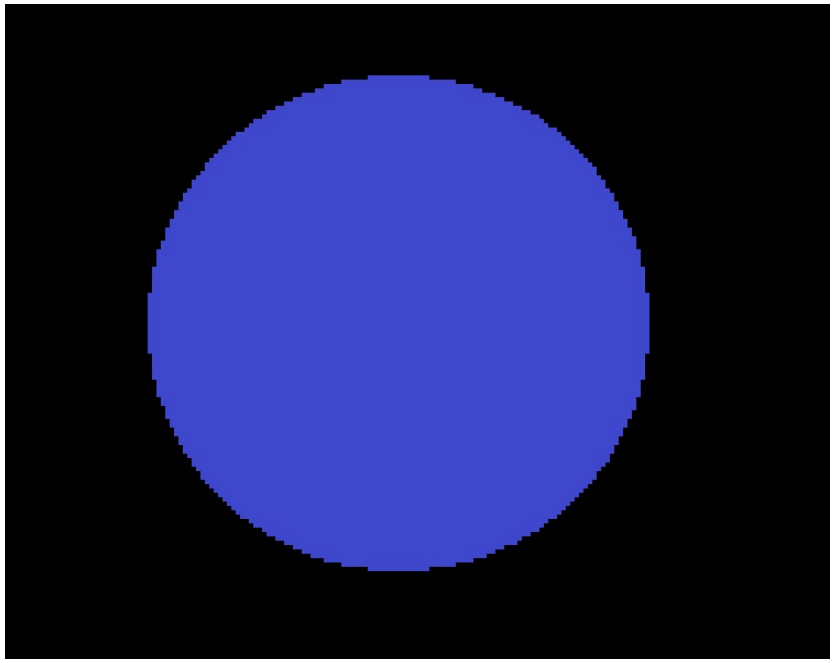


인류 단 50% 만 맞출 수 있는 문제 : **둘 중에 뭐가 3D 일까 ?**

배경지식 – 3D 렌더링 기초

- 3D 그림을 그리기 위해서는 몇 가지 정보가 더 필요한데…

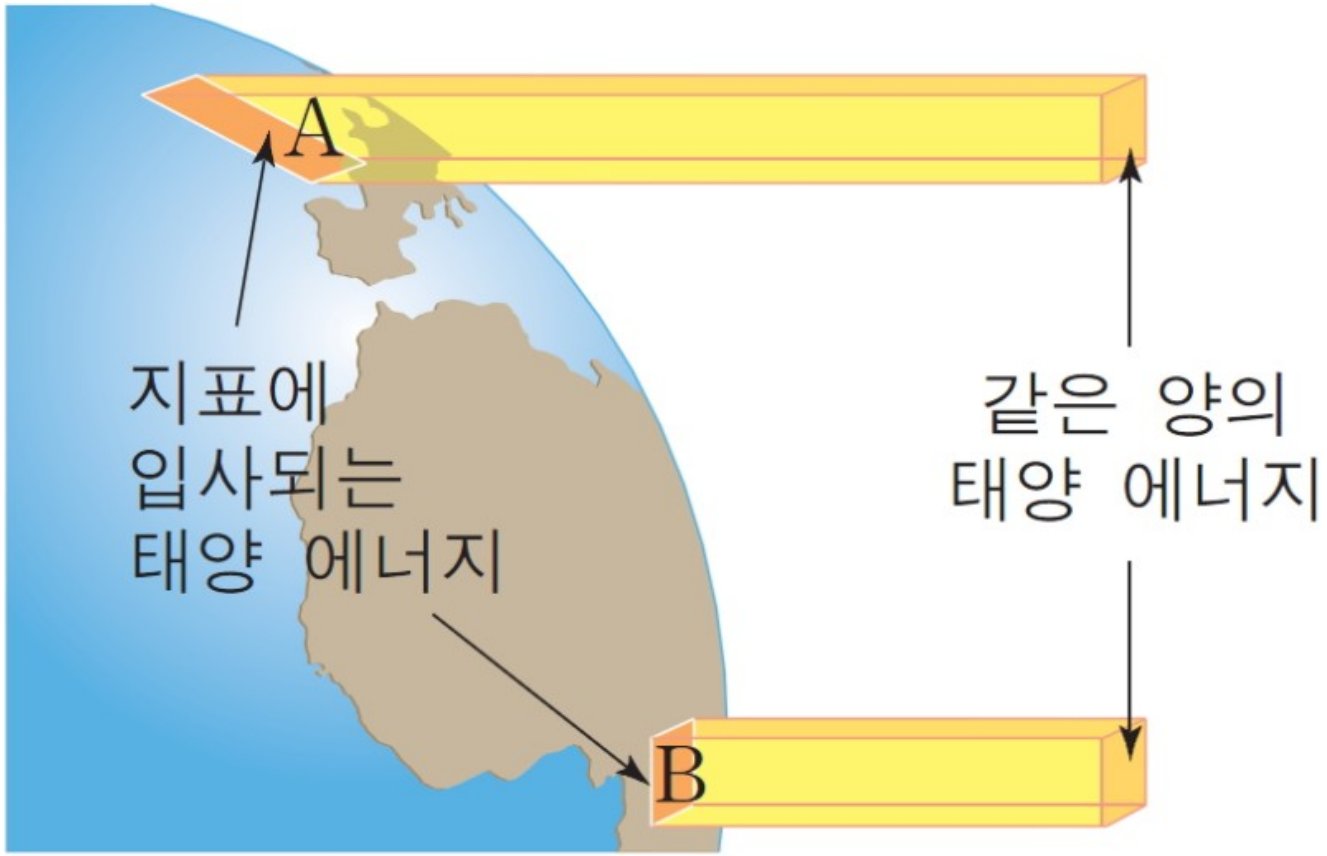




구분이 어렵다 . -> **입체감을 주기위한 추가 정보 필요** **빛과 재질**

배경지식 – 빛 렌더링

- 갑자기 분위기 지구과학



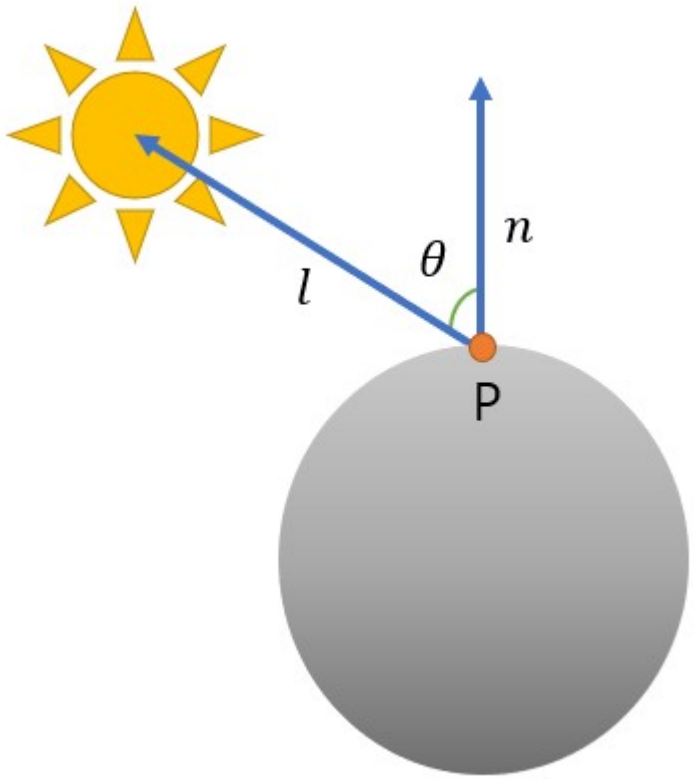
B 가 빛 에너지를 가장 많이 받고

A 가 빛 에너지를 가장 적게 받는다 .

빛을 렌더링하려면 **빛 에너지를 받는 양**을 고려해야한다 .

배경지식 – 빛 렌더링

- 이러한 원리는 대충 이정도로 하면 비슷하지 않을까 ?



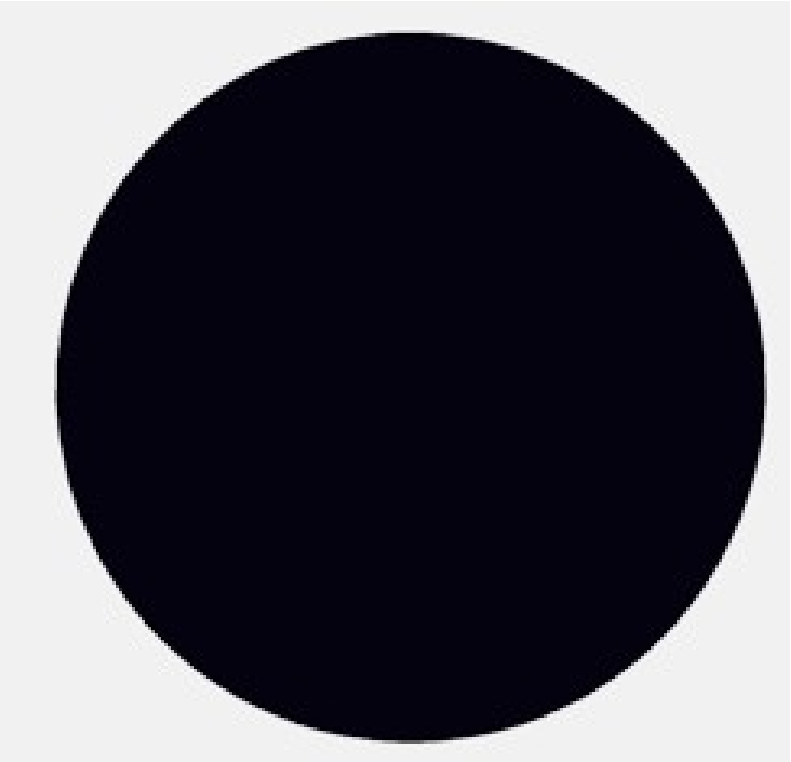
표면의 입사각을 θ 라고 할 때

입사각이 0 일 때 , 빛 에너지가 가장 강하게
입사각이 90’ 일 때 , 빛 에너지가 가장 약하게
대충 cos 함수 사용하면 될 듯

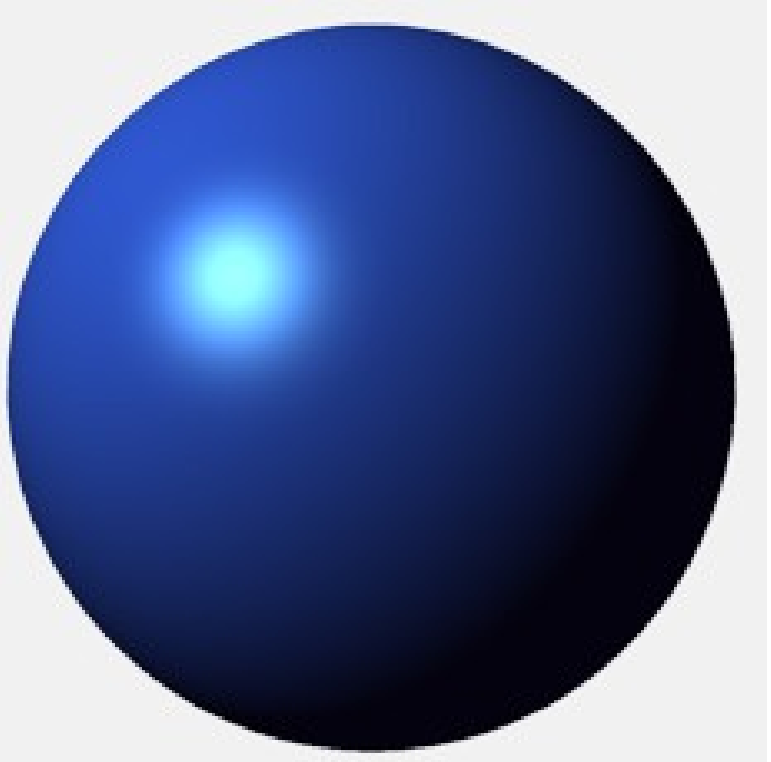
입사각을 구하려면
법선 벡터 (Normal) 이 필요하다 .

배경지식 – 빛 렌더링

- 빛은 입체감을 높인다 .



빛 X



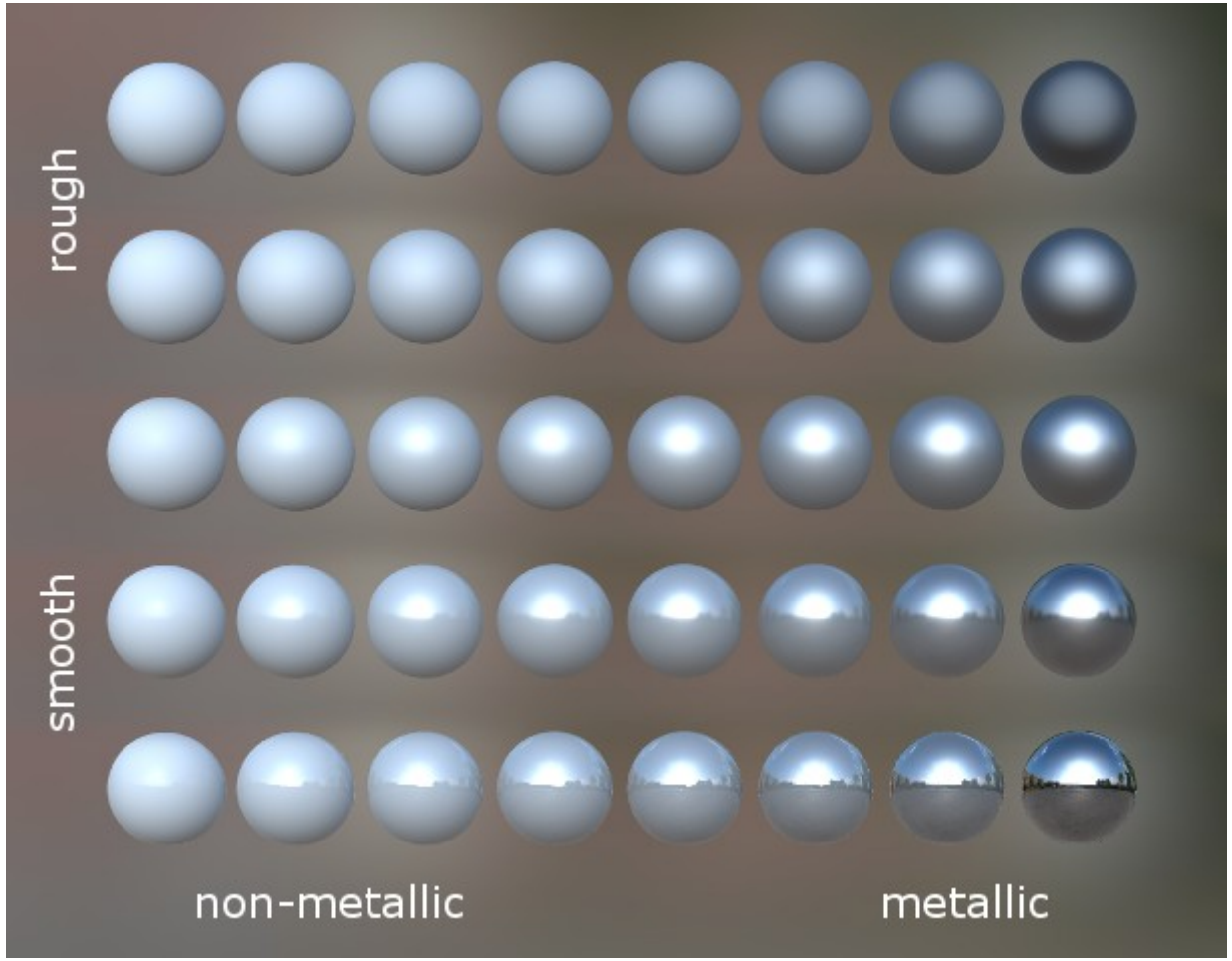
빛 O

힘의 차이가
느껴지십니까 ?

배경지식 – 재질 (Material) 렌더링



배경지식 – 재질 (Material) 렌더링



물체의 재질마다
빛 반사 수식을 디테일하게
설정하기 위해서

Emission (발광정도)
Metallic (금속 재질)
Roughness (거칠기 정도)

등을 빛 반사 공식에 적절히
섞을 수 있음

배경지식 – 재질 (Material) 렌더링

- 언리얼 엔진 4 에서 실제로 사용하는 수식
- 난반사광 계산
- 정반사광 계산

$$f(\mathbf{l},\mathbf{v}) = \frac{\mathbf{c}_{\mathbf{diff}}}{\pi}$$

$$f(\mathbf{l},\mathbf{v}) = \frac{D(\mathbf{h})\,F(\mathbf{v},\mathbf{h})\,G(\mathbf{l},\mathbf{v},\mathbf{h})}{4\,(\mathbf{n}\cdot\mathbf{l})\,(\mathbf{n}\cdot\mathbf{v})}$$

$$D(\mathbf{h}) = \frac{\alpha^2}{\pi\left((\mathbf{n}\cdot\mathbf{h})^2\,(\alpha^2-1)+1\right)^2}$$

$$k = \frac{(Roughness+1)^2}{8}$$

$$G_1(\mathbf{v}) = \frac{\mathbf{n}\cdot\mathbf{v}}{(\mathbf{n}\cdot\mathbf{v})(1-k)+k}$$

$$G(\mathbf{l},\mathbf{v},\mathbf{h}) = G_1(\mathbf{l})\,G_1(\mathbf{v})$$

$$F(\mathbf{v},\mathbf{h}) = F_0 + (1-F_0)\,2^{(-5.55473(\mathbf{v}\cdot\mathbf{h})-6.98316)(\mathbf{v}\cdot\mathbf{h})}$$

결론

아 .. 계산량이 엄청 많구나

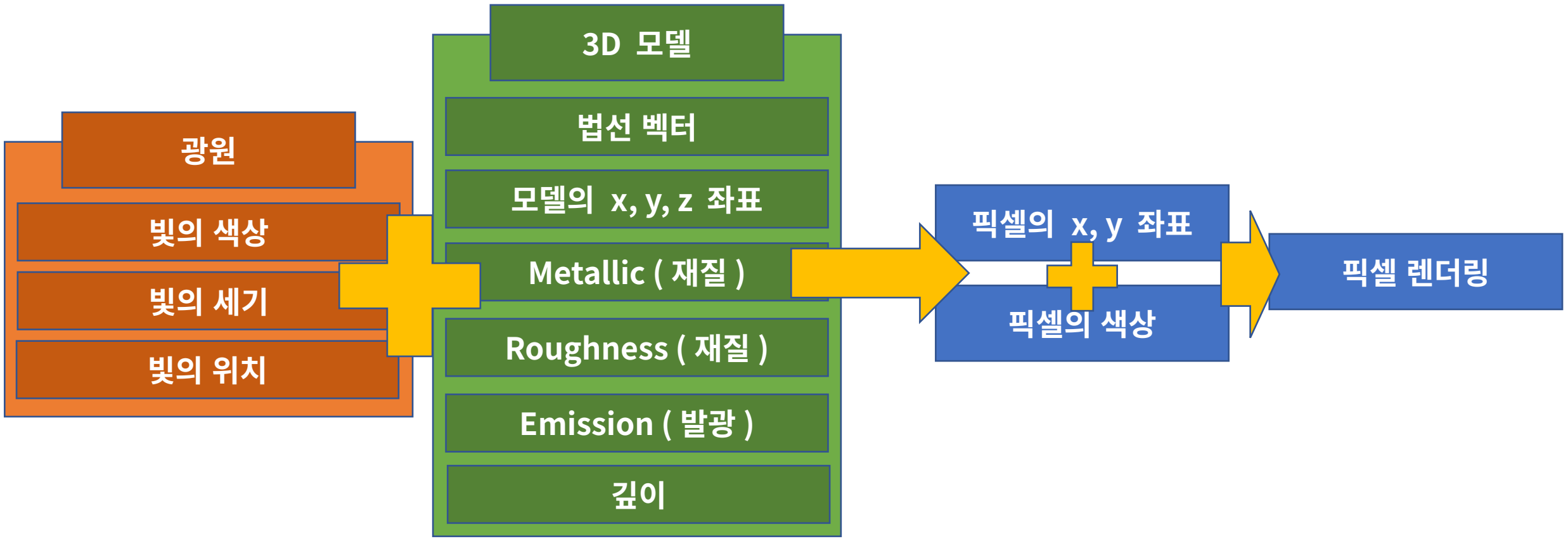
자세한 것은 논문 참고

https://cdn2.unrealengine.com/Resources/files/2013SiggraphPresentationsNotes-26915738.pdf

그래서 3D 물체를 제대로 그리기 위해선 ..

- 픽셀의 **위치**와 **색**을 결정해야한다 .
- **색**을 결정하기 위해서는 **빛 , 재질 , 법선 벡터** 가 필요하다 .
- 더하여 .. 3D 물체들끼리 멀고 가까움을 표현하기 위해 **깊이**가 필요하다 .

그래서 3D 물체를 제대로 그리기 위해선 ..



즉 , 색상을 결정하기 위해서는…



픽셀 하나 당
빛과 재질 수식을 모두 적용해야 함

즉 , 1280 x 720 게임을 실행한다면
앞서 말한 수식을

1 초에 60 번 씩
한 번에 921,600 번 계산해야 함

즉 , 1 초에 최소
55,296,000 번 수식을 계산

전통적인 렌더링 – Forward Rendering

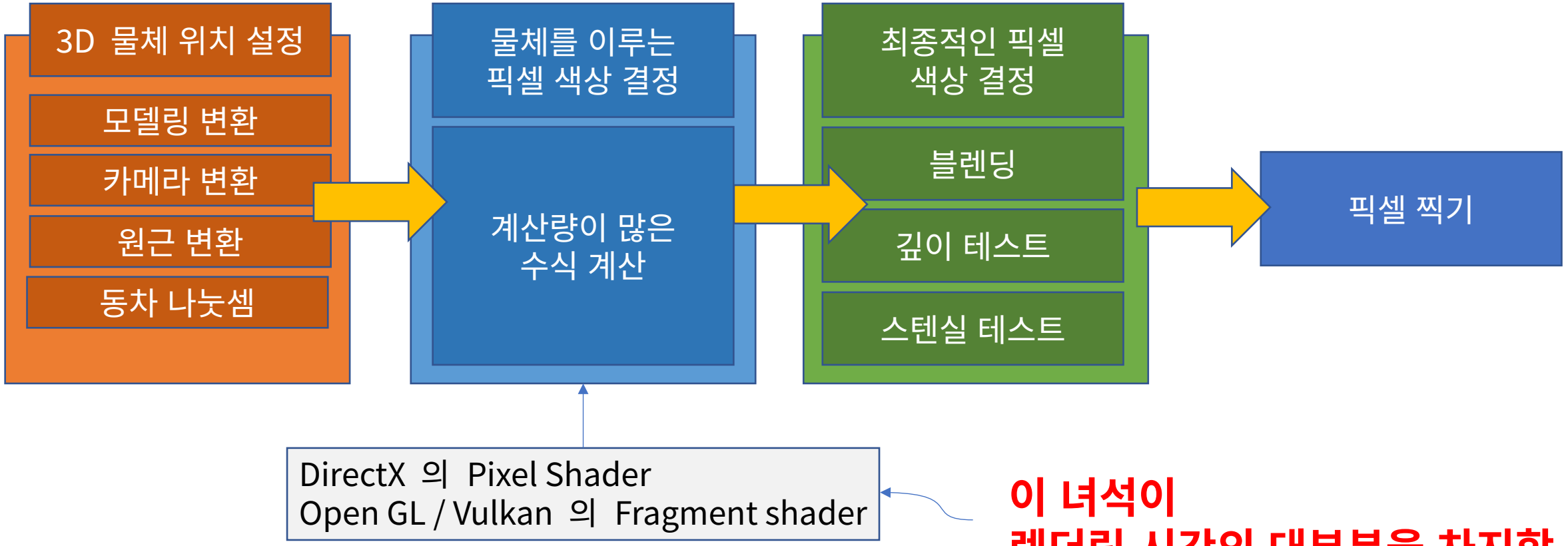
- 혹은 **즉시 렌더링**
- 앞서 말했듯 3D 물체의 렌더링은 다음과 같이 이루어짐

1. 3D 물체의 위치 결정
2. 3D 물체를 이루는 픽셀의 색상 결정 (앞서 설명했던 수식 적용)
3. 최종적인 픽셀 색상을 결정
4. 화면상에 픽셀 찍기

-> 이러한 프로세스를 **렌더링 파이프라인** 이라고 한다 .

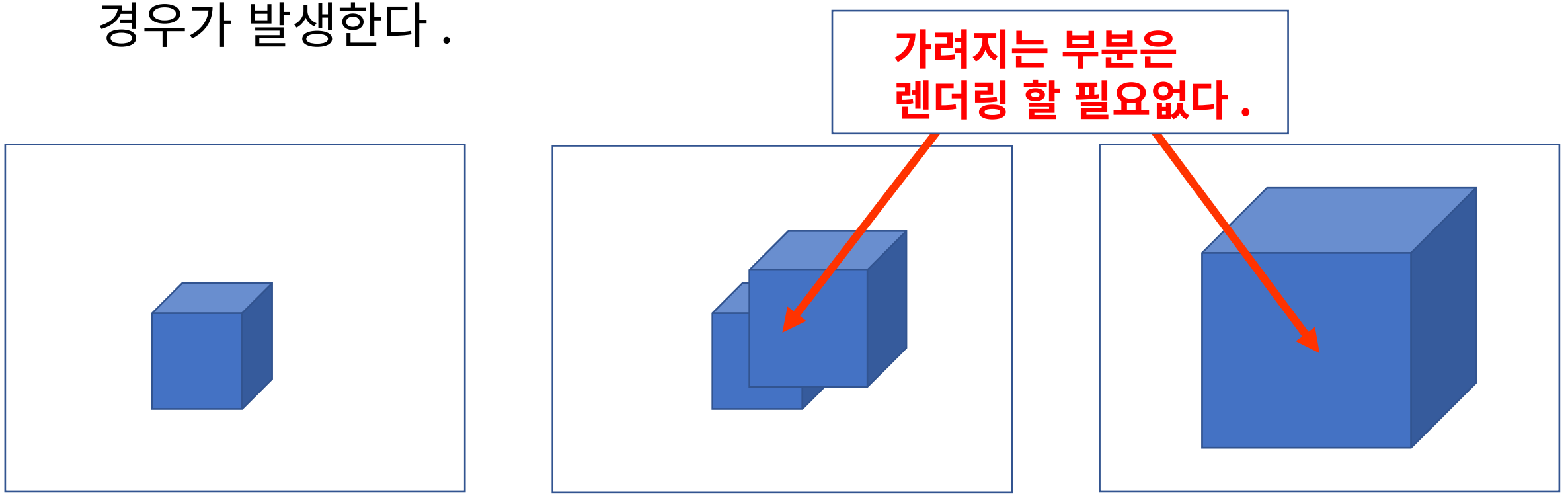
전통적인 렌더링 – Forward Rendering

- 모델이 N 개 있을 때 , 렌더링 파이프라인을 N 개 수행해야 한다 .



Forward Rendering 의 문제점

- 모델마다 렌더링 파이프라인을 수행하기 때문에 , 계산이 낭비되는 경우가 발생한다 .



장면 1

장면 2

장면 3 (최종 장면)

해결책 – Deferred Rendering

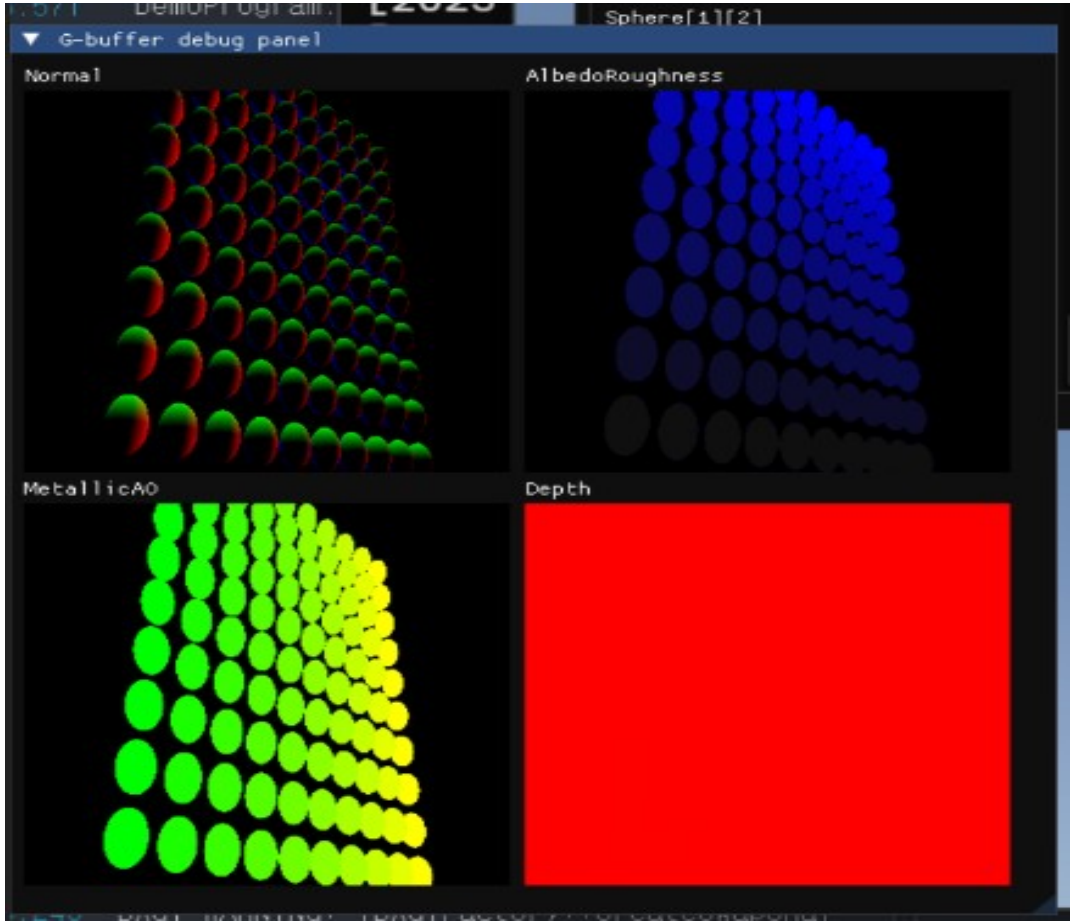
- 모델을 한 번에 렌더링하는 것이 아닌 2 단계로 거쳐서 렌더링한다 .

1. **모든 모델의 기하 정보만 렌더링**
   스크린과 동일한 스크린을 생성해서 , 해당 스크린에 위치 , 법선벡터 , 깊이 , 재질 등과 같은 색상 계산에 필요한 **기하학 정보**만 먼저 렌더링

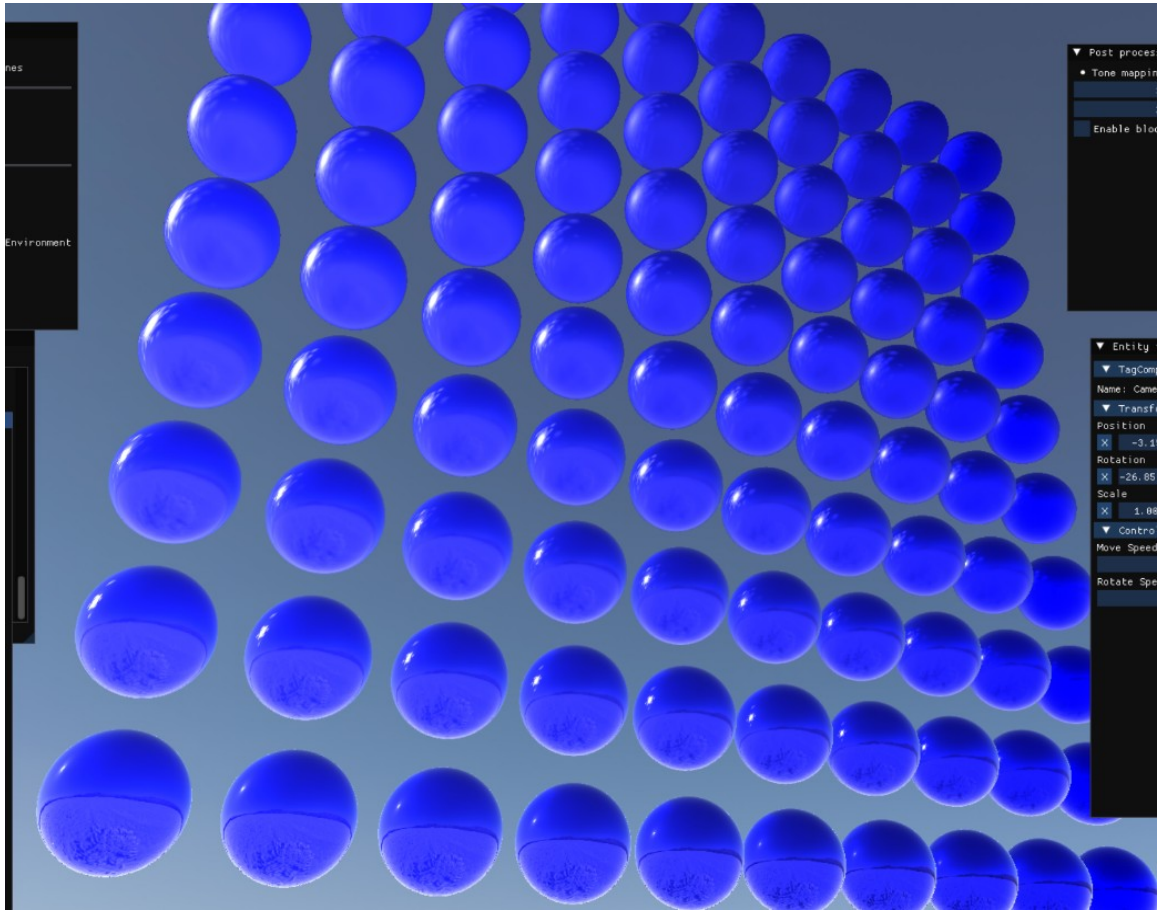
2. **저장된 기하 정보를 토대로 '딱 한 번만' 색상 계산**
   스크린에 저장된 최종적인 기하학 정보만을 이용하여 딱 한 번만 색상을 계산

-> 기하 정보는 동일하게 N 번 렌더링하는데 , 색상 계산한 1 번만 수행함

해결책 – Deferred Rendering



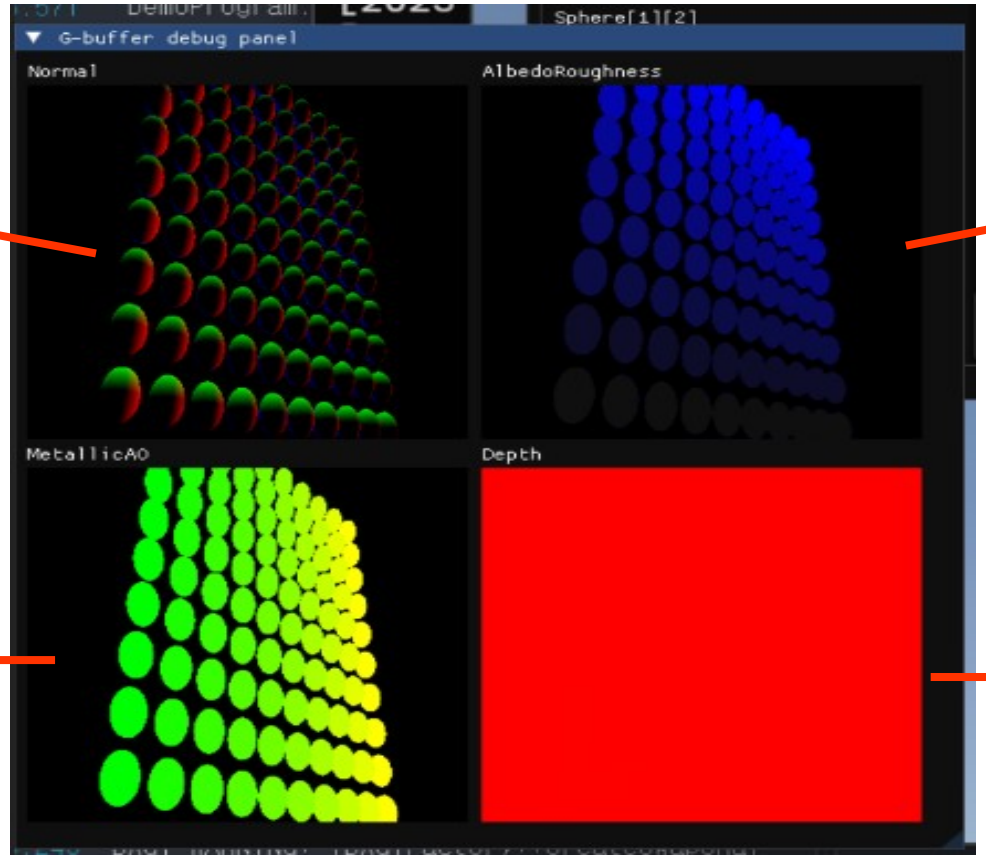
기하 정보들만 렌더링 (g-buffer)



기하 정보를 이용하여 색상 결정

Deferred Rendering – 최적화 기법

법선 벡터
스크린



색상
거칠기
스크린

금속
차폐광
스크린

깊이
스크린

스크린이 N 개면 M 개의 모델을 M*N 번 렌더링 해야함
-> **스크린 항상 최소로 유지하는게 좋음**

Deferred Rendering – 최적화 기법

- 위치를 저장하는 스크린을 따로 두지 않는 이유 ?
  -> 깊이 값에서 위치 값을 복구할 수 있다 .

$$v = v_{ndc} \cdot \quad P^{-1} \cdot \quad V^{-1}$$

$$v_{world} = \frac{v_{xyz}}{v_w}$$

$n_{ndc} = NDC$ 좌표계의 3차원 좌표
P^{-1} = 투영 행렬의 역행렬
V^{-1} = 카메라 행렬의 역행렬
v_{world} = 월드 공간의 3차원 좌표

Deferred Rendering – 최적화 기법

- 스크린의 경우 메모리가 작으면 작을수록 빠름
- **[0, 1] 사이의 값만으로도 충분한 경우**
  ex) LDR 색상 , 금속 , 차폐광 , 거칠기 등 ..

  DXGI_FORMAT_R8G8B8A8_UNORM 사용

- **그 이상의 값이 필요한 경우**
  ex) HDR 색상 , 위치 , 법선 등 ..

  DXGI_FORMAT_R16G16B16A16_FLOAT 사용

  -> 게임에서 32bit float 정도의 정밀도는 필요로 하지 않음

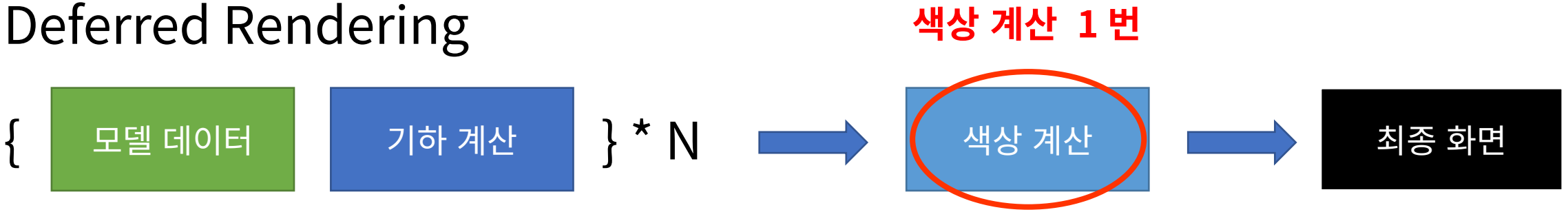
참고 : DirectX 기준

Forward vs Deferred – 파이프라인

Forward Rendering



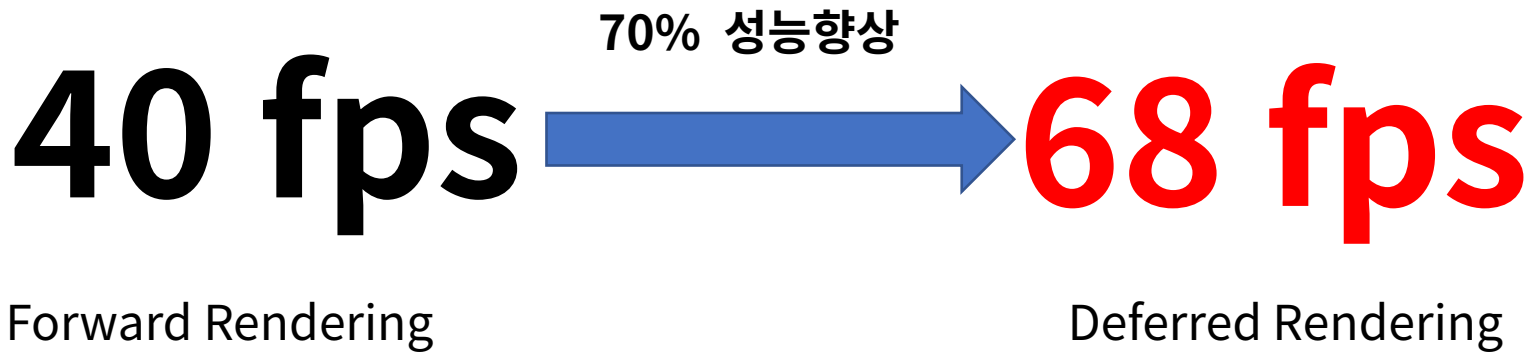
Deferred Rendering



Forward vs Deferred – 스트레스 테스트

- 테스트 환경
  CPU - AMD Ryzen 9 7945HX with Radeon Graphics
  GPU – Nvidia GeForce RTX 4060

  구 100’000 개 렌더링
  총 버텍스 약 6.43 억 개



Deferred Rendering 한계점 및 해결방안

- 안티앨리어싱이 불가능
- 반투명 물체 처리가 불가능

안티앨리어싱

- **앨리어스 (계단 현상) 이란 ?**
  결국 픽셀을 찍는 행위이기 때문에 자세히 보면 픽셀이 상세하게 보이게됨 .
  이를 매끄럽게하는 것이 **안티앨리어싱**

안티앨리어싱 적용

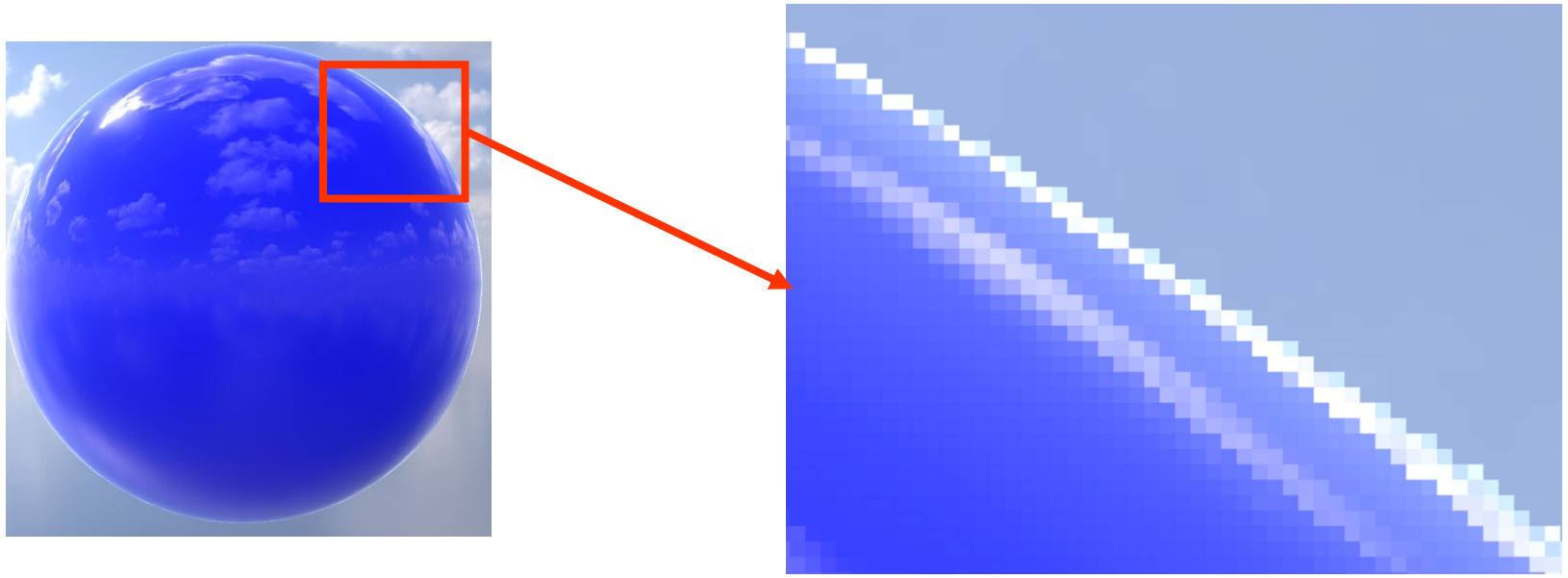




앨리어스 현상

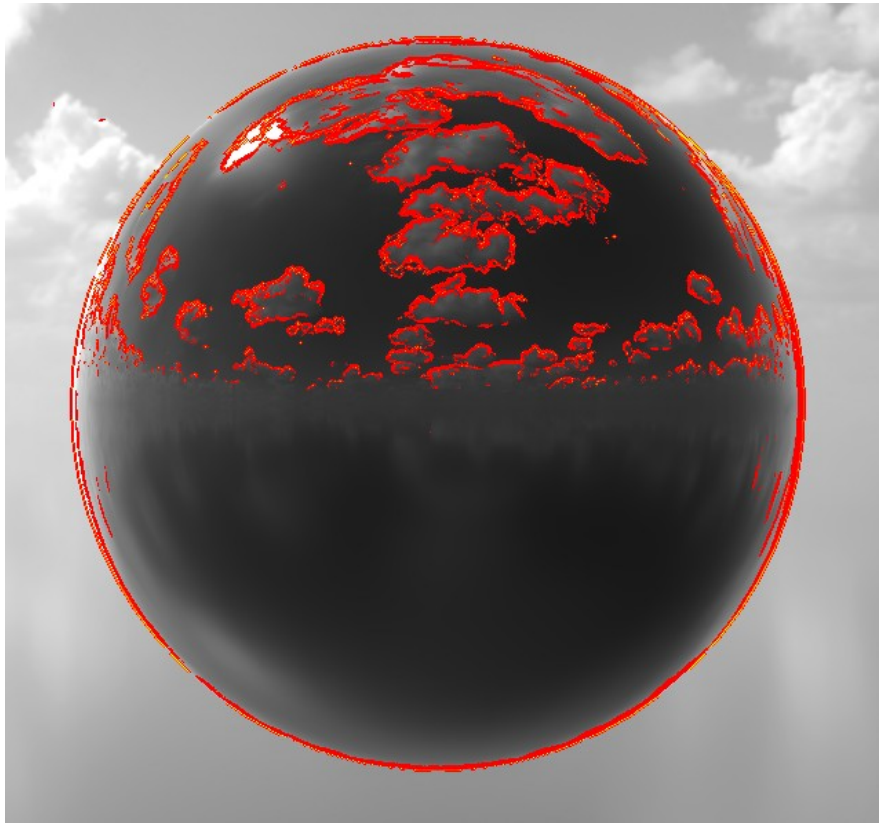
안티앨리어싱

- 일반적으로 하드웨어 차원에서 MSAA (멀티샘플링) 이라는 안티앨리어싱 알고리즘을 지원
- 그러나 , 기하 정보를 저장하는 과정에서 오차가 발생하기에 하드웨어 안티앨리어싱을 사용할 수 없음



안티앨리어싱 – 해결방안

- **직접 안티앨리어싱을 짜면 되는거 아닌가 ?**



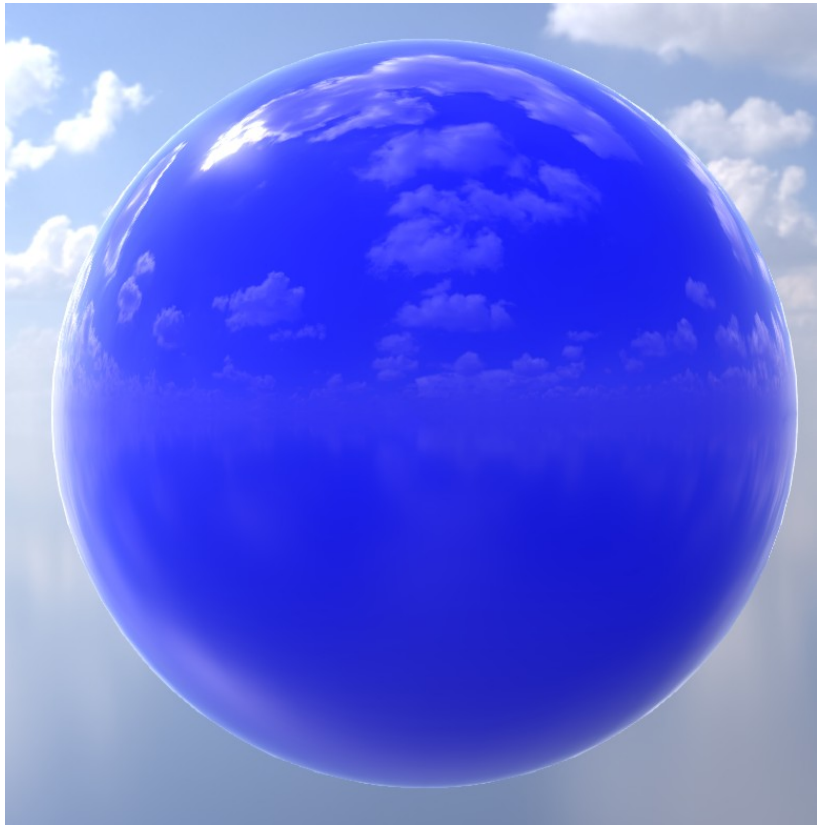
FXAA

1. 색상 차이를 이용하여 경계를 탐색
2. 수직 수평 성분을 분리
3. 엣지의 방향을 이용하여 대비가 높은 픽셀을 선택
4. 오프셋을 계산하여 주변 픽셀과 섞음

자세한 알고리즘은 문서 참고

https://developer.download.nvidia.com/assets/gamedev/files/sdk/11/FXAA_WhitePaper.pdf

안티앨리어싱 – 해결방안



FXAA 의 품질은 MSAA 보다 조금 떨어지지만 성능저하는 0.2% 이하
오히려 MSAA 보다 빠름

-> 좋은 성능으로 만족할 수 있는 품질 제공

TAA 과 같은 고품질 안티앨리어싱도 사용할 수 있음

라스트 오브 어스 2, 사이버 펑크 2077 등 최신게임에서는 TAA 선택

반투명 물체 처리

- 모든 물체를 렌더링하고 색상을 입히는 특성상 반투명 처리가 불가능함



모든 물체를 그리면
이전 기하 정보가 소실되기에

색상을 결정할 때
반투명한 표현이 불가능함

반투명 물체 처리 – 해결방안 1

- **반투명 물체만 따로 모아서 렌더링 (z-sort)**

알고리즘

1. 디퍼드 렌더링을 정상적으로 수행
2. 반투명한 물체들을 모아 깊이를 기준으로 정렬함
3. 깊이가 깊은 것부터 순서대로 포워드 렌더링

문제점

1. 매 프레임 정렬하는 것이 부담스러움 (quick sort 시 n log n)
2. 반투명 물체가 반투명 물체를 관통하는 꼴은 못 그림
   -> 객체단위 정렬이기 때문

반투명 물체 처리 – 해결방안 2

- **디더링을 이용하여 근사적으로 반투명 구현**



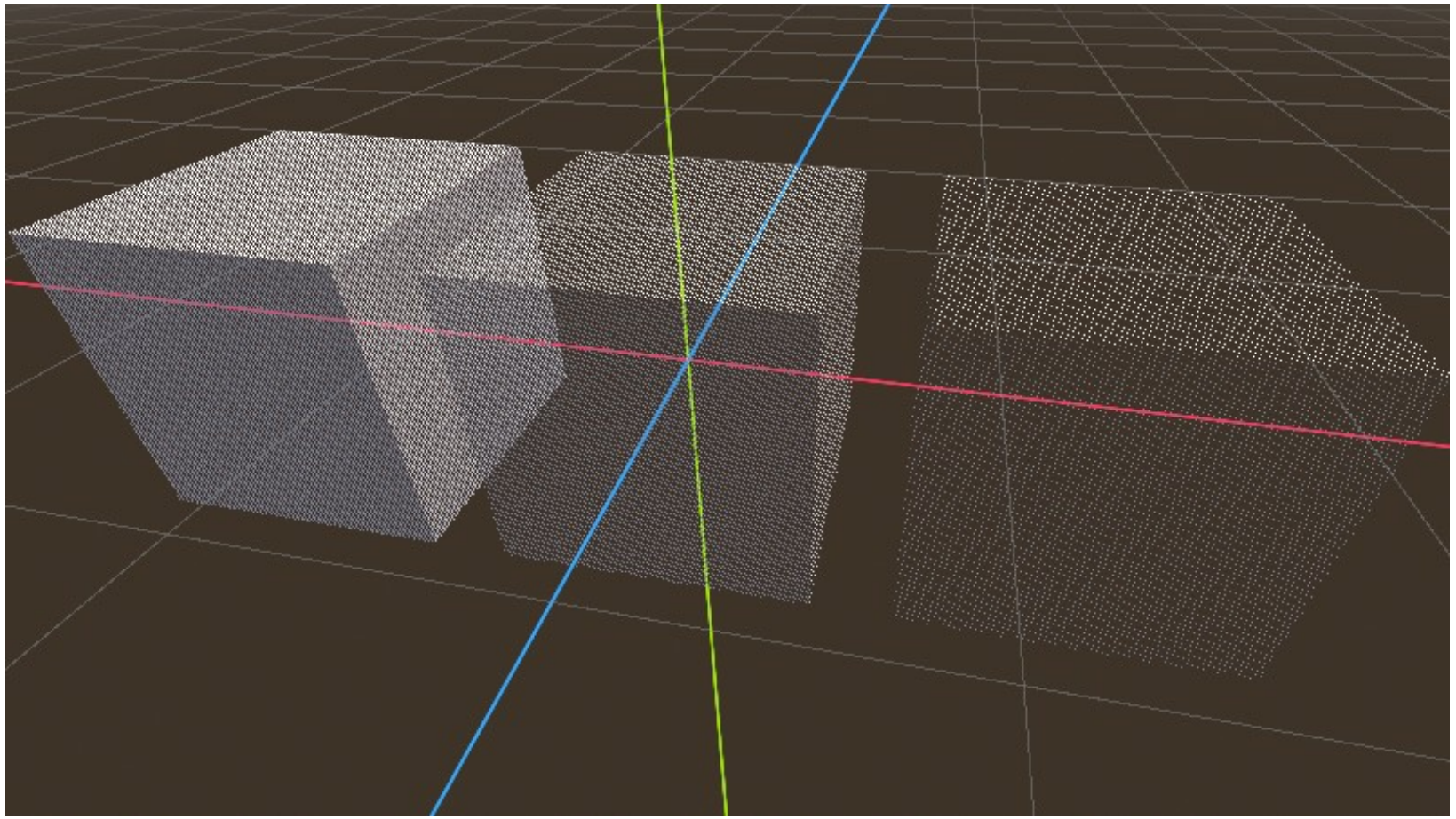
디더링은 제한된 색으로 명암을 나타내는 기법

좌측 사진의 경우 흰색과 검정 만으로 명암을 표현

투명한 물체 -> **디더링을 강하게**
불투명한 물체 -> **디더링을 약하게**

반투명 효과를 근사적으로 구현할 수 있음

반투명 물체 처리 – 해결방안 2



충분히 그럴듯한 효과 !

반투명 물체 처리 – 해결방안 2

- **디더링을 이용하여 근사적으로 반투명 구현**

장점

그럴듯한 품질과 매우 뛰어난 성능
디퍼드 렌더링으로 한 번에 처리 가능

슈퍼마리오 오디세이 , 세키로 등 현대적인
AAA 게임에서도 선택하는 기법

단점

진정한 의미의 반투명은 아님



반투명 물체 처리 – 해결방안 3

- Order-Independent Transparency

  GPU 안에 자료구조를 유지하여 픽셀의 기여도를 병합하는 방법

  장점
  진정한 의미의 투명 렌더링 알고리즘
  정렬이 필요없음
  픽셀 단위로 투명을 처리할 수 있음

  단점
  성능 감소가 여전히 존재함
  구현이 어려움

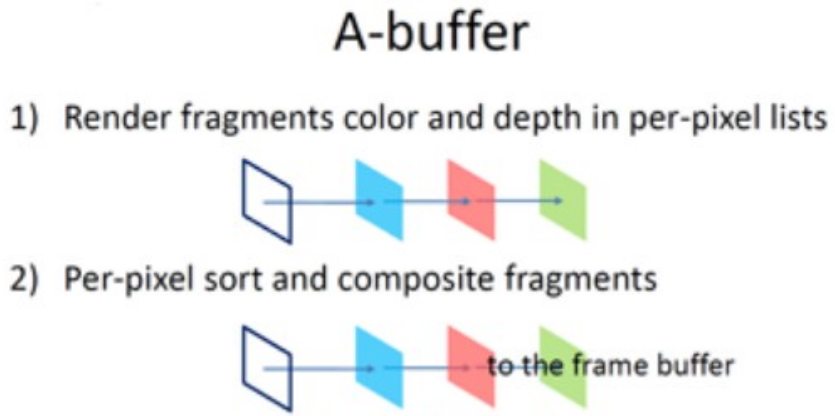


그림 2. A-버퍼 루틴

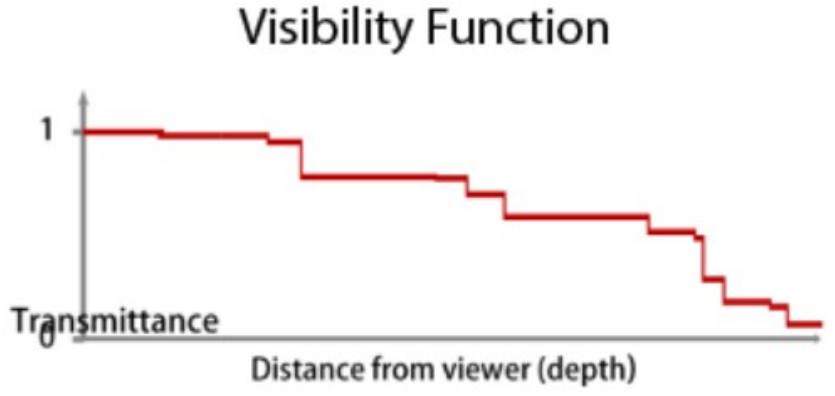


그림 3: 가시성 기능

반투명 물체 처리 – 해결방안 3

자세한 내용은 발표 주제를 넘어서기에 자세한 것은 문서 참조

https://www.intel.com/content/www/us/en/developer/articles/technical/oit-approximation-with-pixel-synchronization.html

Q/A